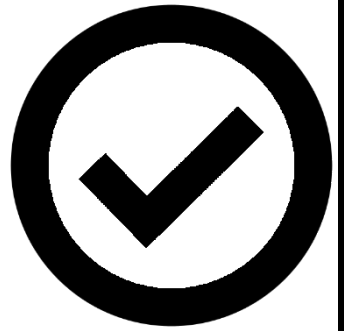


**BUSINESS
PROFESSIONALS**
of **AMERICA**
Giving Purpose to Potential



C# PROGRAMMING (330)

STATE 2021

PRODUCTION PORTION:

Program 1: Dynamic Telephone Security Keypad _____ (480 points)

TOTAL POINTS _____ (*480 points*)

Test Time: 90 minutes

Dynamic Telephone Security Keypad

In this exercise, you will create a C# Windows Form Application that creates a telephone keypad that has a dynamic allocation of letters and numbers. This keypad is a future ready security pad and is used for entering randomly generated passwords: the key's sequences and locations are never the same. The user can press on the keys to enter in a random password given by the program to verify it works. The keys values will be imported from a text file, and every time the user clicks on a key, the color temporarily changes and a system sound is played. A message box is displayed to inform the user if they entered in the password correctly. Then the user is given the option to reset the keys and password.

Requirements:

1. You must create a C# Windows Form Application named CS_330_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CS_330_01_2345_6789.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. The main form must *not* have minimize or maximize buttons in the Form Handle.
4. The main form must have text "Telephone Keypad" for its title.
5. Your form must have a table layout panel with a 3x5 layout (3 columns and 5 rows).
6. The panel must have a background color of "Deep Sky Blue" and all of the cell border styles need to be set to inset.
7. The cells for the panel must be of equal size *except* for the bottom row which contains the submit button and two text boxes (one shows the temp password and the other will allow the user to enter in their password). See the sample pictures below.
8. Add a label to every cell *except* for the bottom row; the label should fill the cell with the Dock option under properties. The text should be aligned to the middle. The label must have the same back color as the panel.
9. Program each label in the panel to have its text set to a random letter or number from the text file called "letter.txt" (note this file only contains lowercase letters and the digits 0-9. The values in the file are separated by commas. The last value has **NO** comma at the end.
 - a. No label in the panel can have the same letter or digit for a given run of the program.
 - b. The text in the label must be color Red.
 - c. There should be a click event handler for the labels called "keypadClick".
 - d. When the user clicks on a label, its color needs to temporarily change to the color Black and revert back to its original color of Red (this must be visible so you will need to slow it down).
 - e. When clicked the system sound "Exclamation" should be played.
10. A random password (6 characters only) is created based upon values placed in the text of the labels. The password is displayed in the text field in column 2 row 5 (see sample picture).
11. A timer should be used to control how fast a user can click the labels:
 - a. Set the interval to be 750 (note interval of 1000 is 1 second).
 - b. It should only be enabled once a user clicks on a label so set it to a default of "False".

- c. When the program starts (or restarts) use a time control feature to so the text being placed in each label with an interval 100 and its original color (Black) switches to Red; this is used to control the placement of text into the labels.
 - d. Note: the timer should be started when a label is clicked (interval 750) and when the program places text in each label at the beginning (interval 100).
12. After the password has been entered by clicking on the labels and the submit button has been clicked:
- a. If the passwords match the message should be displayed: "Your passwords match!", "Congratulations on security keypad prototype!"
 - b. If the passwords do *not* match the message is displayed: "INCORRECT", "Passwords do *not* match."
 - c. Next, the user should be given a Yes No message box: "Do you want to try again?", "Yes or No?"
 - i. If yes: the program resets and a new random set of label text appears, and a new random password is created.
 - ii. If no: form application closes.
 - d. **WARNING:** the password must be entered by clicking on the labels.

Sample Pictures:

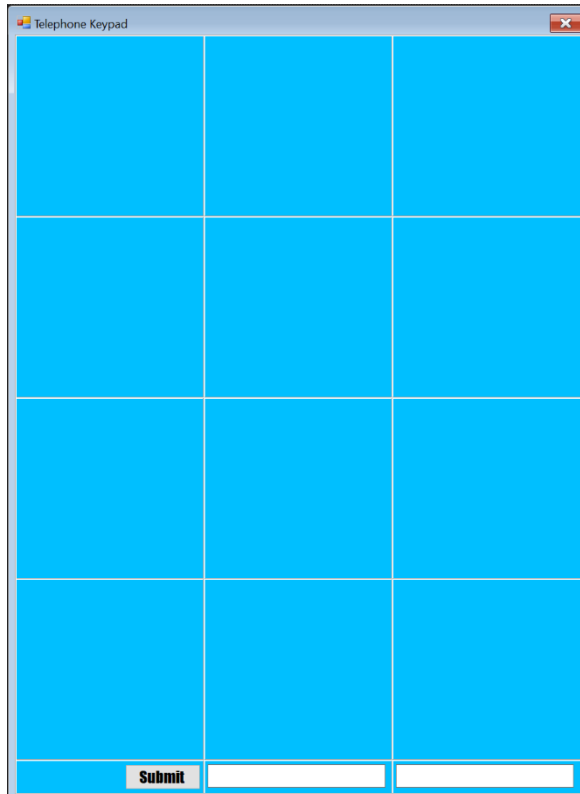


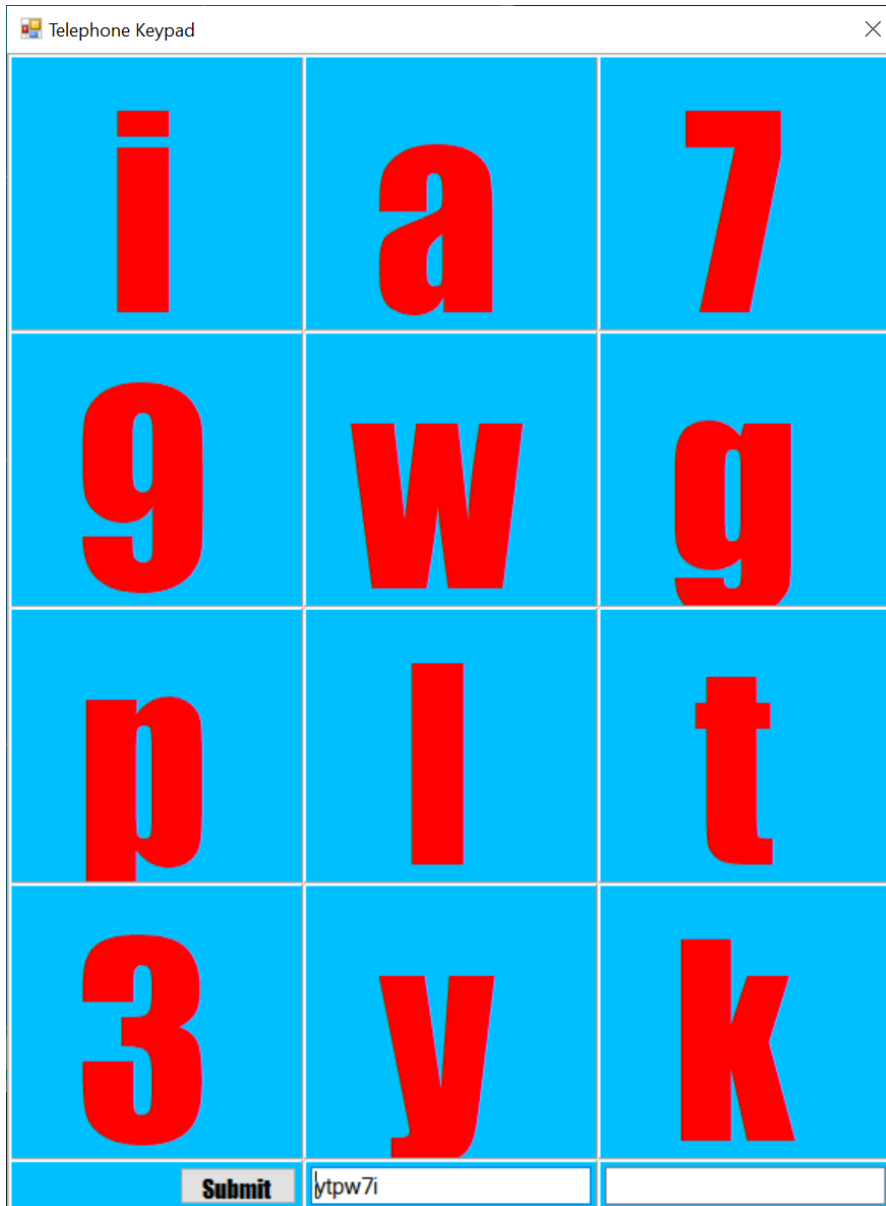
Figure 1: Panel layout. Notice the bottom row has a “Submit” button and two text boxes.



The image shows a portion of a web application. It features a blue header bar. Below it, there is a row of three elements: a grey button labeled 'Submit', a white text box containing the text 'ytpw7i', and another empty white text box. Arrows from the captions below point to these elements.

Figure 2a: Randomly created password appears here.

Figure 2b: User enters their password in other text box via pressing the label. **WARNING:** manually entering text will not result in a proper comparison. You must enter the password via the labels.



The image shows a window titled 'Telephone Keypad'. It contains a 4x3 grid of buttons, each with a red character on a blue background. The characters are: Row 1: 'i', 'a', '7'; Row 2: '9', 'w', 'g'; Row 3: 'p', 'l', 't'; Row 4: '3', 'y', 'k'. At the bottom of the window, there is a 'Submit' button, a text box containing 'ytpw7i', and an empty text box.

Figure 3: Randomly selected letter and numbers from the “letter.txt” file populate the labels.

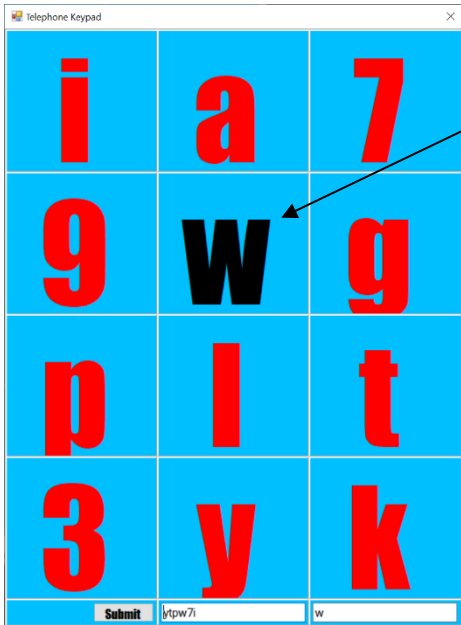


Figure 4: When a label is clicked, the color changes to Black. And then changes back to Red after an interval of 750.

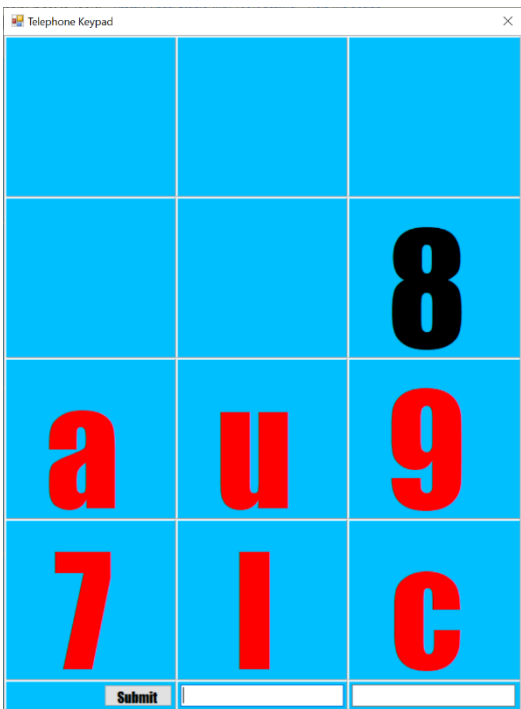
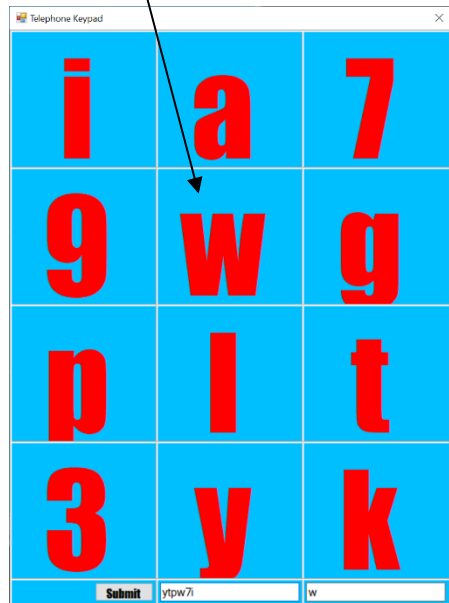
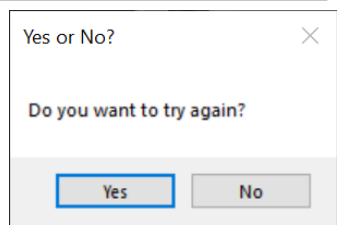
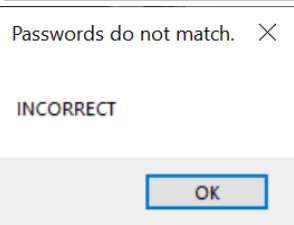
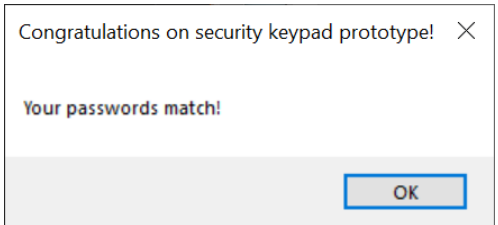


Figure 5: When the program starts the labels are populated with random text from letter.txt file. This example starts, from the bottom up. Each label gets text at an interval of 100, they start as Black, and then is set to Red. It should do this when the program resets, too.

Figure 6: These are the message box windows that appear after the password has been entered by clicking on the buttons. When no is selected, the program closes.



You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on any work you submit for grading. Remember to use your contestant # where appropriate.

Submit a copy your entire solution/project to the flash drive provided. You must submit your entire solution/project so that the graders may open your project to review the source code and/or build and execute your solution/project. **Submissions that do not contain source code will not be graded.**

Development Standards

- Standard name prefixes must be utilized for variables.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).

Your application will be scored on the following criteria:

Solution and Project

The project is present on the flash drive _____ 10 points
The project is named according to the naming conventions _____ 10 points

Program Execution

Code copied to USB drive and program runs from USB _____ 20 points

If the program does not execute, then the remaining items in the program executive section receive a score of zero.

The title on the form is “Telephone Keypad” _____ 10 points
The layout panel is 3 columns and 5 rows _____ 10 points
The layout of the form has equal sized labels for the text and matching colors per the description _____ 10 points

The main form does not have minimize, or maximize buttons in the Form Handle _____ 20 points

When the form loads it randomly places in the labels with NO repeating values _____ 20 points

The temporary password in the bottom text field matches text selected from the distributed text placed in the labels _____ 30 points

The temporary password in the bottom text field only has 6 characters _____ 20 points

When program starts there is an interval of 100 that slows the populating of the labels with text, and the colors of the labels are black and switch to red _____ 30 points

There is an interval of 750 between clicking events on the labels _____ 30 points

A system sound is played when labels are clicked _____ 10 points

A message box displays when correct/matching password is entered _____ 20 points

A message box displays when incorrect/not-matching password is entered _____ 20 points

A message box displays asking user if they want to restart the program _____ 20 points

Program restarts successfully _____ 20 points

Source Code Review

Code is commented at the top, for each function, and as needed _____ 10 points

Code uses reasonable and consistent variable naming conventions _____ 10 points

Code to import the letter.txt file and split by comma is present _____ 30 points

Code for keypadClick () is present _____ 20 points

Timer code is present to control clicking _____ 20 points

Code for checking random password and user entered password is present _____ 20 points

Code for assigning random text to labels is present _____ 10 points

Code for creating random 6 character password is present _____ 20 points

Code is present to delay the placement of text in labels and switches color black to red with interval of 100 _____ 30 points

Total Points: 480 points

SOURCE

```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10 using System.IO;
11
12 namespace TelephoneKeypad
13 {
14     public partial class Keypad : Form
15     {
16
17         string[] values;
18         List<string> valueList = new List<string>();
19         Random random = new Random();
20         Label initialClick = null;
21
22         string passwordAutoFill = "";
23         string passwordCheck = "";
24         int tempCounter = 0;
25
26         //Begins program
27         public Keypad()
28         {
29             InitializeComponent();
30             listReader();
31             AssignKeysToButtons();
32         }
33
34
35         //Reads comma separated text file and imports as array, converts to
36         List
37         public void listReader()
38         {
39             System.IO.StreamReader myReader;
40             myReader = new System.IO.StreamReader(@"C:\Users\Ryan Patton
41             \OneDrive\FISD\BPA\C Sharp RWP\letter.txt");
42             string line = myReader.ReadToEnd();
43             myReader.Close();
44
45             values = line.Split(',');
46
47             foreach (string a in values)
48             {
49                 valueList.Add(a);
50             }
51         }
52     }
53 }
```

```
48         }
49
50
51     }
52
53     //Resets the entire contents. Note: we keep it simple by just reading  ➤
54     //from the text file again
55     private void resetVariables()
56     {
57         listReader();
58         passwordAutoFill = "";
59         passwordCheck = "";
60         tempCounter = 0;
61         textBox1.Text = "";
62         textBox2.Text = "";
63     }
64
65
66
67     //Assigns the key values a random value from imported list and removes  ➤
68     //it
69     //Only one value can appear, no duplicates
70     private async void AssignKeysToButtons()
71     {
72         foreach(Control control in tableLayoutPanel1.Controls)
73         {
74             Label keyLabel = control as Label;
75             if(keyLabel != null)
76             {
77                 int randomNumber = random.Next(valueList.Count);
78
79                 keyLabel.Text = valueList[randomNumber];
80                 tempCounter++;
81
82                 //This is how we get our random password
83                 if (tempCounter%2 == 0)
84                     passwordAutoFill += keyLabel.Text;
85
86                 await Task.Delay(100);
87                 keyLabel.ForeColor = Color.Red;
88                 valueList.RemoveAt(randomNumber);
89             }
90         }
91         textBox1.Text = passwordAutoFill;
92     }
93
94     //Controls the clicking of the keypad
```

```
95         //and plays a system media sound.
96     private void keypadClick(object sender, EventArgs e)
97     {
98         if (timer1.Enabled == true)    //this slows down the click speed
99             return;
100
101         var sound = System.Media.SystemSounds.Exclamation;
102         sound.Play();
103
104         Label clickedKey = sender as Label;
105         passwordCheck += clickedKey.Text;
106         textBox2.Text = passwordCheck;
107
108         if (clickedKey != null)
109         {
110             if(clickedKey.ForeColor == Color.Black)
111             {
112                 return;
113             }
114
115             if(initialClick == null)
116             {
117                 initialClick = clickedKey;
118                 initialClick.ForeColor = Color.Black;
119
120                 timer1.Start();
121                 return;
122             }
123         }
124
125     }
126 }
127
128 //Timer is used to control the display
129 private void timer1_Tick(object sender, EventArgs e)
130 {
131     timer1.Stop();
132     initialClick.ForeColor = Color.Red;
133
134     initialClick = null;
135 }
136
137
138 //Checks the password. WARNING: the password is only checked based upon the label click events.
139 //Manual entry of the password is not handled with this program.
140 private void checkPassword(object sender, EventArgs e)
141 {
142
```

```
143
144
145     if (passwordCheck.CompareTo(passwordAutoFill) == 0 )
146     {
147         MessageBox.Show("Your passwords match!", "Congratulations on ↗
            security keypad prototype!");
148         DialogResult responseYN = MessageBox.Show("Do you want to try ↗
            again?", "Yes or No?", MessageBoxButtons.YesNo);
149         if (responseYN == DialogResult.Yes)
150         {
151             resetVariables();
152             AssignKeysToButtons();
153         }
154         else { Application.Exit(); }
155     }
156     else
157     {
158         MessageBox.Show("INCORRECT", "Passwords do not match.");
159         DialogResult responseYN = MessageBox.Show("Do you want to try ↗
            again?", "Yes or No?", MessageBoxButtons.YesNo);
160         if(responseYN == DialogResult.Yes)
161         {
162             resetVariables();
163             AssignKeysToButtons();
164         }
165         else { Application.Exit(); }
166     }
167 }
168 }
169 }
170 }
171
```